

안상운(27) Cloud/Infrastructure Engineer

Contact.

Web Portfolio: portfolio.cloudwoon.com

Blog : <https://velog.io/@luckyprice1103/posts>

Github : github.com/luckyPrice

소개

안정적이고 효율적인 인프라 운영에 강점을 가진 클라우드/시스템 엔지니어입니다. AWS, GCP 등의 클라우드, 자동화, 모니터링 환경 구축을 경험하며 서비스 운영 안정성을 높여왔습니다. 더 나은 아키텍처와 운영 환경을 설계해 사용자와 개발자가 신뢰할 수 있는 인프라를 만들어가는 엔지니어가 되고자 합니다.

기술 스택

- 클라우드: AWS, GCP, Azure
- **IaC (Infrastructure as Code)**: Terraform, Ansible
- 데이터베이스: MySQL, Oracle, AWS RDS
- 가상화 & OS: Linux, VMware, VirtualBox
- 시스템 운영 & 모니터링: Prometheus, Grafana, AWS CloudWatch
- **CI/CD & 컨테이너**: Docker, Kubernetes, Jenkins, GitHub Actions, GitLab, ArgoCD
- 프로그래밍 언어: Python, Java

프로젝트

카카오테크 부트캠프 - On The Top (5개월, 6명)

2025.04 ~ 2025.07

On The Top은 사용자의 현재 책상 사진을 기반으로 추가하면 좋을 데스크테리어 아이템을 추천해 주고, 해당 사진을 기반으로 추천된 아이템이 적용된 모습을 **AI**를 통해 생성하여 제공하는 서비스입니다. 커뮤니티를 즐길 수 있고, 포인트로 물건들을 구매할 수 있습니다.

[성과]

- **Kubernetes** 워커 노드에 스팟 인스턴스를 적용하여 안정성을 유지하며 인프라 비용 **50% 이상 절감**
- **Terraform(IaC), Ansible** 기반 자동화를 통해 **GCP**에서 **AWS**로의 서버 이전을 **1일** 만에 완료
- 단일 인스턴스에서 **Kubernetes** 기반 **3-Tier** 아키텍처까지 단계적으로 설계 및 고도화
- 서버 간 통신 취약점을 자체적으로 발견 및 증명하고, **Nginx/WAF**를 통한 해결 및 구조적 개선 제안

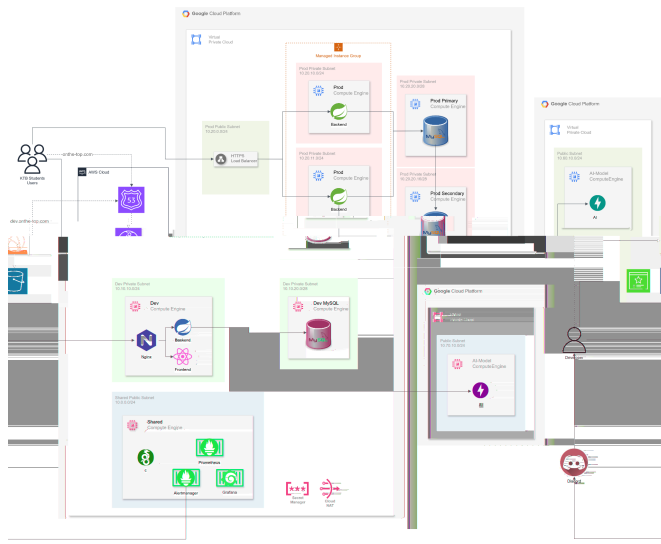
1. 주도적인 기술 문화 도입 및 효율적인 커뮤니케이션

- **WHY? 문서** 문화 기획 및 도입: "당연한 기술은 없다"는 원칙 아래, 모든 기술 선택의 근거를 문서화했습니다. 이를 통해 팀원들의 질문, 내부 발표, **PL** 미팅 등에서 명확한 백데이터를 기반으로 기술적 의사소통의 효율성과 정확성을 크게 향상시켰습니다

2. 단계별 아키텍처 설계 및 인프라 고도화

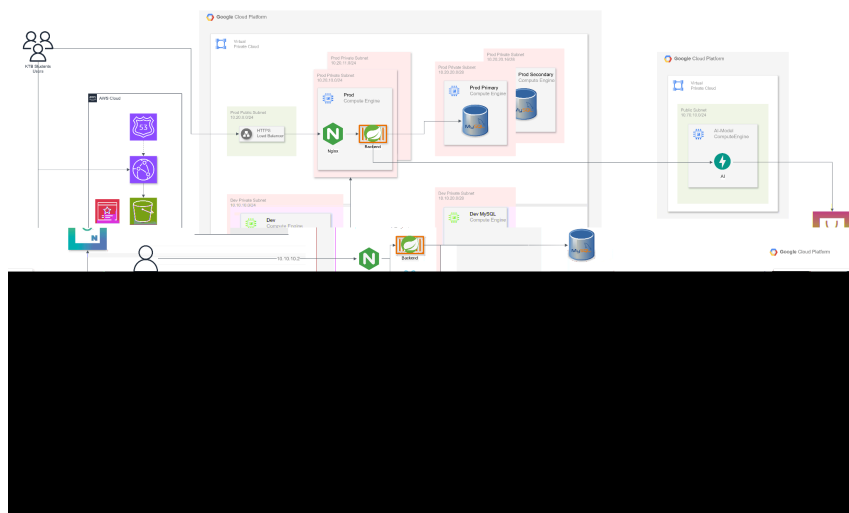
서비스의 안정성과 확장성을 확보하기 위해 단일 인스턴스 배포에서 시작하여 **CI/CD, 3-Tier, Docker, Kubernetes**를 순차적으로 도입하며 아키텍처를 점진적으로 발전시켰습니다. 각 단계의 전환을 통해고가용성(**HA**) 확보, 수평 확장, 무중단 배포, 유연성 확보 등 핵심적인 아키텍처 원칙들을 체득하고 실제 구현 경험을 쌓았습니다.

3-Tier 아키텍처



기존의 전통적인 3-Tier 구조를 도입하여 프레젠테이션, 애플리케이션, 데이터 계층의 역할을 명확히 분리했습니다. 운영 환경에서는 GCP의 **Managed Instance Group**과 이중화된 **MySQL** 데이터베이스, **HTTPS Load Balancer**를 활용해 고가용성을 확보했습니다. 또한, **AWS S3**와 **CloudFront**를 연동하여 정적 콘텐츠 전송을 최적화하는 멀티 클라우드 환경을 구축했습니다. **GitHub Actions**를 도입해 빌드 자동화 파이프라인을 구축하고, **Prometheus, Grafana**를 활용한 모니터링 시스템과 **VPN**을 통한 보안 체계를 마련하여 안정적인 서비스 운영 기반을 다졌습니다.

Docker를 활용한 환경 일관성 확보



서버 환경 불일치와 의존성 관리의 어려움을 해결하기 위해 **Docker**를 도입했습니다. 개발, 테스트, 운영 환경을 코드로 관리하고 격리함으로써 환경의 일관성을 확보했으며, 특히 **Docker Compose**를 활용해 다중 컨테이너 애플리케이션의 실행과 관리를 자동화했습니다. 무중단 배포를 위해 **Nginx** 리버스 프록시 기반의 **Blue-Green** 배포 환경을 직접 구축했고, 문제 발생 시 즉시 이전 버전으로 되돌릴 수 있는 롤백(**Rollback**) 기능을 구현하여 서비스 안정성을 극대화했습니다.

EKS 기반의 클라우드 네이티브 전환

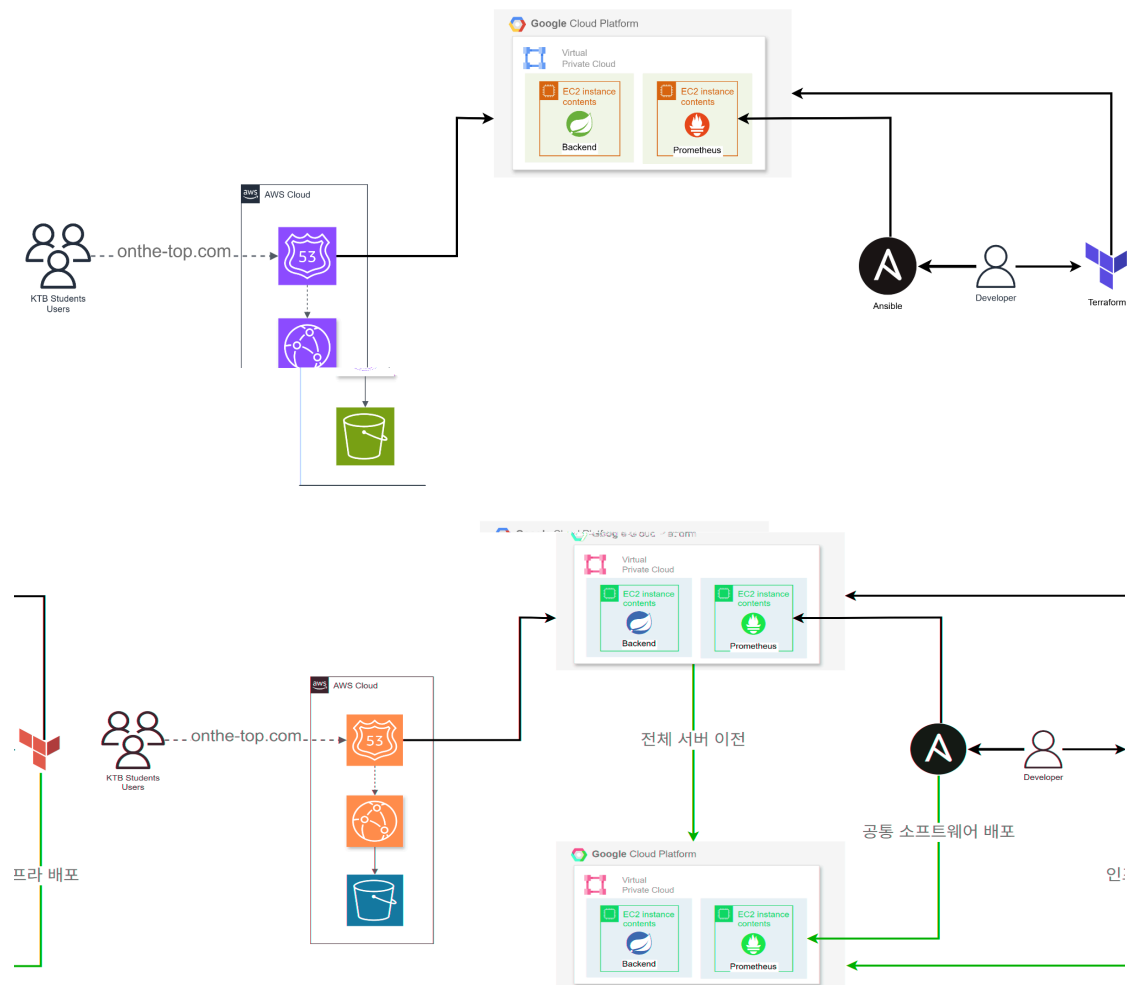
(06 2024)

활용의
점

대용량 데이터 처리 및 분석을 위한 클라우드 네이티브 아키텍처를 구축하고, 기존 온프레미스 인프라에서 클라우드 네이티브 아키텍처로 전환하는 과정에서 EKS 기반의 컨테이너 오케스트레이션 서비스를 채택해 인프라 관리 복잡도를 낮추고 개발팀이 애플리케이션에 집중할 수 있도록 했습니다. 컨테이너 자동 복구(**Self-healing**) 및 오토 스케일링(**Auto-scaling**) 기능을 활용하여 장애 발생 시 수동 개입 없이 복구되고, 트래픽에 따른 리소스 탄력적 운영으로 비용 효율성을 높였습니다. **CI/CD 파이프라인** 도입

자동화된 환경 속에서도 시스템의 근본적인 한계점을 발견했습니다. AI 서버의 **OOM(Out-of-Memory)** 장애가 전체 서비스 중단으로 이어지는 구조적 결함을 분석한 결과, 그 원인이 동기식 구조의 강한 결합에 있음을 확인했습니다. 단순히 서버 사양을 높이는 대신, **Redis** 기반의 메시지 큐를 도입해 시스템을 비동기 구조로 재설계했습니다. 이를 통해 특정 시스템의 장애가 다른 시스템으로 전파되는 것을 차단하고 데이터 무결성을 확보하여 더욱 탄력적인 시스템을 구축할 수 있었습니다.

3. 클라우드 비용 최적화 및 마이그레이션 전략



- 멀티클라우드(**AWS/GCP**) 기반 마이그레이션 아키텍처 설계: 프로젝트 초기부터 **GCP** 크레딧 소진에 대비하여 이전 가능성을 전제로 한 유연한 아키텍처를 설계했습니다. 컴퓨팅 자원은 **GCP** 프로젝트 단위로, 도메인(**Route53**) 등 핵심 자원은 **AWS**에 고정하여 클라우드 환경 간 의존성을 최소화했습니다.

- **laC** 및 자동화 기반의 클라우드 이전: **Terraform(laC)**과 **Ansible**을 활용해 인프라 구성을 코드로 완벽히 관리했습니다. 이를 통해 예상치 못한 오류 발생 시에도 신속한 수정 및 복구가 가능한 환경을 구축하여, **1일** 만에 성공적으로 서버 이전을 완료했습니다.
- **Spot** 인스턴스를 활용한 **Kubernetes** 비용 최적화: **Kubeadm** 기반 클러스터의 워커 노드를 **Spot** 인스턴스로 구성하고, 자동 재조인(**re-join**) 스크립트를 구현하여 안정성을 확보했습니다. 이를 통해 워커 노드당 약 **69%**의 비용을 절감했으며, 전체 인프라 비용의 **50%** 이상을 절약하는 성과를 거두었습니다.

4. 선제적인 아키텍처 취약점 분석 및 보안 강화

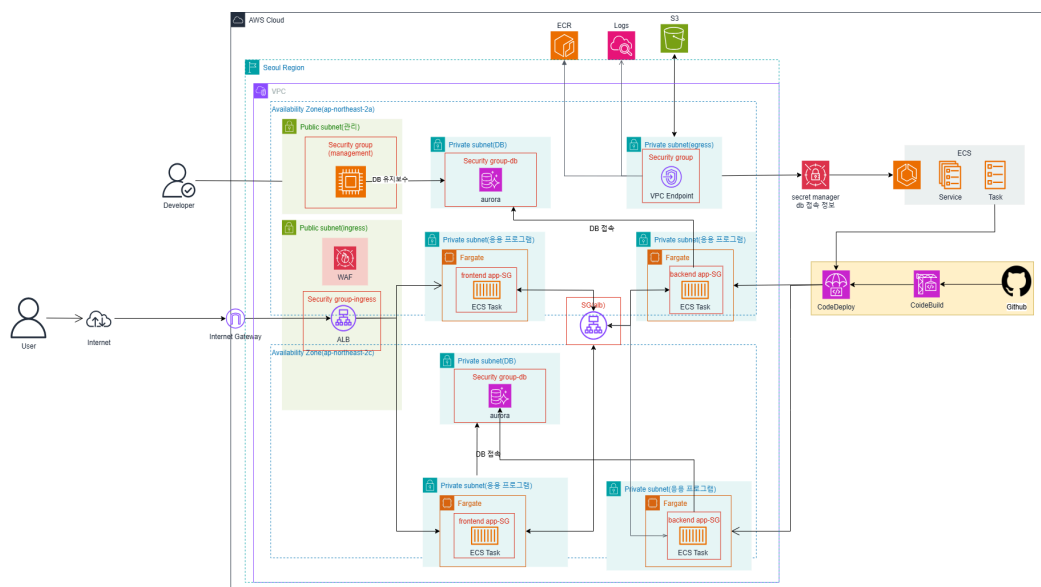
- 서버 스푸핑 취약점 발견 및 증명: 내부 **IP**를 사용하는 서버 간 통신에서 발생할 수 있는 스푸핑(**Spoofing**) 가능성을 이론적으로 분석하고, 실제 **dev** 서버 대상 공격 테스트를 통해 취약점을 증명했습니다.

다. **awscli**를 통한 보안 인식 제고: **Nginx**, **Bé** 에
 / 증 위 □ /x)

[성과]

- 서버리스 컨테이너 환경 구축: **ECS Fargate** 기반으로 서버 관리 부담 없는 인프라를 구축하여 운영 오버헤드를 **90%** 이상 감소시키고 개발 생산성을 향상
- **CI/CD** 파이프라인 완전 자동화: **CodePipeline** 기반 파이프라인으로 **GitHub Push**부터 무중단 배포까지 자동화하여 배포 주기를 수동 대비 **80%** 단축
- 다계층 보안 아키텍처 적용: **WAF, Security Group** 등을 유기적으로 구성하여 웹 공격 및 비인가 접근을 원천 차단하는 심층적인 보안 체계 확립
- 안정적인 **API** 연동: **Spring Boot**와 **React** 간의 **REST API**를 설계 및 구현하여 서비스의 핵심 기능들을 안정적으로 연결

1. AWS 서버리스 컨테이너 기반의 인프라 설계 및 구축



- **1-1. ECS on Fargate**를 활용한 서버리스 환경 구축: EC2 인스턴스 프로비저닝, OS 패치, 스케일링 등 서버 관리에 드는 운영 부담을 없애기 위해 **ECS Fargate**를 도입했습니다. 이를 통해 개발팀이 인프라 걱정 없이 비즈니스 로직 개발에만 집중할 수 있는 고효율 개발 환경을 조성했습니다.
- **1-2. Amazon Aurora** 기반의 고가용성 데이터베이스 운영: 데이터의 안정성과 가용성을 극대화하기 위해 관리형 서비스인 **Amazon Aurora**를 도입했습니다. 자동 장애 복구(**Failover**) 및 백업 기능을 활용하여 데이터베이스 관리의 복잡성을 크게 줄이고, 서비스 연속성을 확보했습니다.

2. CI/CD 파이프라인 완전 자동화 및 무중단 배포 실현

- **2-1. AWS Code** 시리즈를 활용한 **End-to-End** 자동화: **CodePipeline**, **CodeBuild**, **CodeDeploy**를 연동하여 GitHub Push 이벤트 발생 시, Docker 이미지 빌드부터 ECR 저장, ECS 배포까지의 전 과정을 완전 자동화했습니다. 이를 통해 휴먼 에러를 제거하고 배포 안정성을 100% 확보했습니다.
- **2-2. Blue/Green** 배포 전략을 통한 무중단 서비스: **CodeDeploy**의 **Blue/Green** 배포 기능을 적용하여, 신규 버전(**Green**)의 안정성을 완전히 검증한 후 트래픽을 전환하는 방식으로 배포 중단 시간을 '0'으로 만들었습니다. 배포 실패 시에도 즉시 이전 버전(**Blue**)으로 롤백이 가능하여 서비스 안정성을 극대화했습니다.

3. 다계층 보안 적용 및 안정적인 운영 환경 구축

- **다계층 방어(Defense-in-Depth)** 보안 체계 확립: **AWS WAF**를 통해 SQL Injection, XSS 등 7개종의 웹 공격을 차단하고, **Security Group**으로 IP/Port 기반의 엄격한 접근 제어를 적용했습니다. 또한 **VPC Endpoint**를 활용하여 AWS 서비스 간 통신을 안전하게 관리하여 보안 수준을 극대화했습니다.

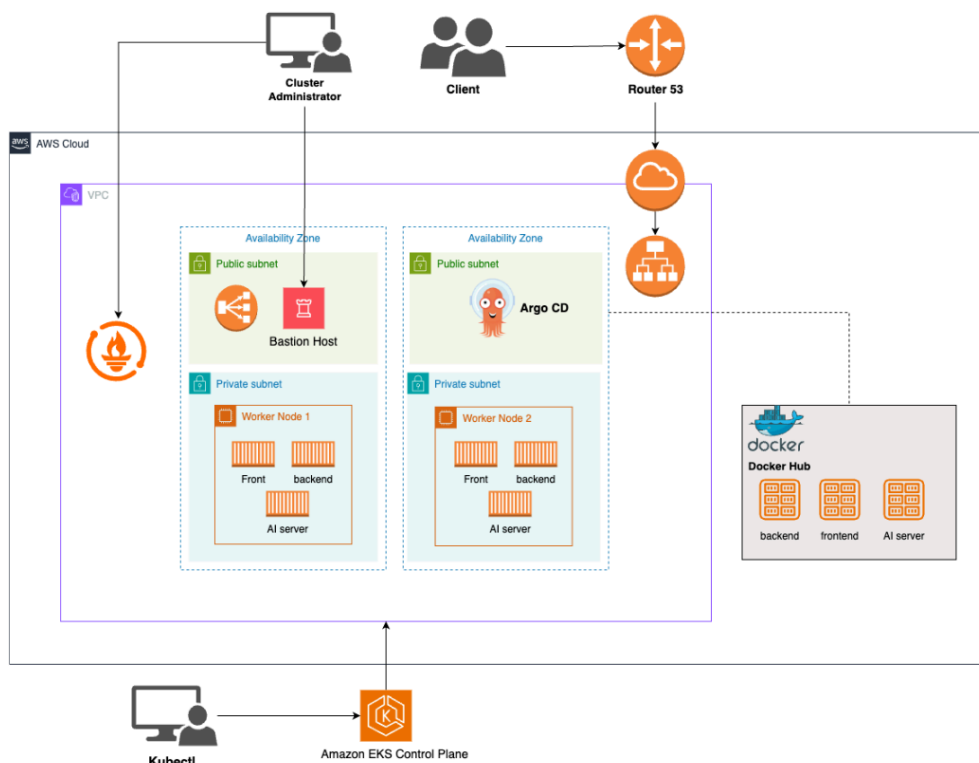
2개월)

2025.01 ~ 2025.02

LLM(거대 언어 모델)을 활용해 사용자가 작성한 개발 학습 기록(TIL)을 **Markdown** 형식으로 자동 변환하고, **GitHub, Notion** 등 외부 플랫폼에 원클릭으로 업로드하는 생산성 향상 웹 서비스입니다.

[성과]

- **Kubernetes** 네이티브 **GitOps** 환경 구축: **Jenkins, ArgoCD, EKS**를 조합하여 **Git**을 단일 정보 소스로 사용하는 효율적인 **GitOps** 워크플로우를 도입하고 배포 안정성을 확보
- 클라우드 네이티브 모니터링 시스템 구축: **Prometheus**와 **Grafana**를 연동하여 EKS 클러스터의 핵심 메트릭을 시각화하고, 장애 예측 및 선제적 대응이 가능한 체계 마련
- 통신을 위한 **API** 설계: **FastAPI, Nest.js, React** 등 이기종 프레임워크 간의 효율적인 통신을 위해 REST API를 설계하고 구현



1. Kubernetes 네이티브 GitOps CI/CD 파이프라인 구축

- **GitOps** 워크플로우 도입: **ArgoCD**를 도입하여, **Git Repository**의 **Manifest** 파일을 단일 정보 소스(**Single Source of Truth**)로 삼는 **GitOps** 워크플로우를 구축했습니다. 이를 통해 배포의 일관성과 명확성을 확보하고, 모든 인프라 변경 이력을 **Git**에서 투명하게 관리했습니다.
- **Jenkins**와 **ArgoCD**를 연동한 배포 파이프라인: **Jenkins**는 **GitHub Push**를 감지하여 **Docker** 이미지를 빌드하고 **Docker Hub**에 **Push**하는 역할(**CI**)을, **ArgoCD**는 **Git**의 변경 사항을 **EKS** 클러스터에 동기화하는 역할(**CD**)을 담당하도록 역할을 명확히 분리하여 효율적인 파이프라인을 완성했습니다.

2. EKS 기반의 탄력적인 컨테이너 인프라 운영

- **Amazon EKS**를 활용한 클러스터 관리 효율화: 쿠버네티스 컨트롤 플레인의 설치 및 운영 부담을 **AWS**에 위임하는 관리형 서비스 **EKS**를 채택하여, 인프라 관리가 아닌 애플리케이션 운영에 집중할 수 있었습니다.
- 오토 스케일링을 통한 비용 효율성 극대화: 트래픽에 따라 **Pod** 수를 조절하는 **HPA(Horizontal Pod Autoscaler)**와 **Worker Node** 수를 조절하는 **Cluster Autoscaler(CA)**를 함께 적용했습니다. 이를 통해 트래픽이 적을 때는 리소스를 최소화하고, 많을 때는 안정적으로 서비스를 제공하는 비용 효율적인 인프라를 구축했습니다.

3. 클라우드 네이티브 모니터링 및 Alerting 시스템 구축

- **Prometheus**와 **Grafana** 기반의 가시성 확보: **Prometheus**를 통해 **EKS** 클러스터의 모든 컨테이너와 노드의 핵심 메트릭(**CPU**, **Memory** 등)을 수집하고, **Grafana** 대시보드를 통해 직관적으로 시각화하여 시스템 상태를 한눈에 파악할 수 있도록 했습니다.
- 장애 예측을 위한 선제적 **Alerting** 체계 마련: 단순 모니터링을 넘어, 리소스 임계치 초과 등 이상 징후 발생 시 **Slack**으로 자동 알림을 보내는 **Alerting** 규칙을 설정했습니다. 이를 통해 장애가 발생하기 전에 문제를 인지하고 선제적으로 대응할 수 있는 안정적인 운영 체계를 갖추었습니다.

https://github.com/luckyPrice/Til_project

교육 및 학습

한성대학교 컴퓨터공학과 학사 과정

2018.03 ~ 2025.02

- 클라우드 컴퓨팅, 데이터베이스, 운영체제, 알고리즘, 데이터통신, 선형대수, 모바일&스마트시스템, 웹프로그래밍, 객체지향언어, 컴퓨터 구조, 웹서버프로그래밍, 네트워크프로그래밍, 설계패턴, 고급 모바일 프로그래밍, 웹프로그래밍, 자료구조, 오픈소스소프트웨어 등 수료

카카오테크 부트캠프 클라우드 네이티브 과정 수료.

2025.02 ~ 2025.08

최신 클라우드 네이티브 기술 스택(컨테이너, 쿠버네티스, CI/CD)뿐만 아니라, 서버 운영, 네트워크 구성, 보안 정책, 모니터링과 장애 대응 등 인프라 엔지니어에게 필요한 전반적인 역량을 함께 다루며 인프라 설계부터 애플리케이션 배포, 자동화, 운영까지 전 과정을 아우르는 실무 중심의 전문가 양성 과정입니다.

- 클라우드 인프라: AWS, Azure 기반 인프라 설계 및 운영
- 컨테이너 & 오케스트레이션: Docker, Kubernetes 활용 애플리케이션 배포 및 운영
- CI/CD: GitHub Actions, Jenkins를 통한 자동화된 배포 파이프라인 구축
- 서버 관리: Linux 서버 운영, Windows 서버 기본 관리, 사용자·권한 관리
- 네트워크 & 보안: VPC, 서브넷, 방화벽, SSL 인증서, 보안 그룹 설정
- 인프라 자동화: Terraform 기반 IaC(Infrastructure as Code) 실습
- 모니터링 & 로그 분석: Prometheus, Grafana 활용 환경 구축 및 장애 대응
- 실무 프로젝트: 팀 단위 클라우드 아키텍처 설계 및 운영 프로젝트 수행

블로그를 통한 학습 정리

2021.05 ~

공부한 내용을 스스로 정리하며 지식을 체화하기 위해 기술 블로그를 운영하고 있습니다. 주로 **AWS**, 클라우드 인프라, 여러 개발 관련 실습 과정과 오류 해결 과정, 개념 정리 등을 기록하고 있습니다.

Link : <https://velog.io/@luckyprice1103/>

자격증

2023.11

정보처리기사

2024.04

SQLD

2024.11

AWS Certified Solutions Architect Associate

2025.02

OPIc IH